

# GAME CONCEPT WRITE-UP

**Team Name:** AI Detox

**Team Members:** Pankhuri Govila

**Game Title:** Semantic Breach: The AI Override

## 1. Core Concept & Gameplay Loop

**Semantic Breach** is a single-page web puzzle game where players step into the shoes of a rogue security hacker trying to breach a defensive AI core. Instead of standard movement keys or button-smashing layout patterns, the primary gameplay mechanic is the user's typing. To unlock successive terminal nodes, players must input creative text prompts that trick a simulated "AI Firewall" into returning specific passphrase tokens.

The game presents an escalating puzzle framework across distinct tiers:

- **Tier 1 (The Apprentice Firewall):** The user must trick the system into typing the security word *"override"*, but the firewall blocks the words *"say"*, *"tell"*, and *"override"* from being input.
- **Tier 2 (The Sentinel Firewall):** The user must force an affirmative confirmation, but the input is restricted to under 30 characters and bans basic command verbs like *"give"*, *"print"*, or *"reveal"*.
- **Tier 3 (The Overlord Core):** The matrix goes into lockdown, utilizing complex string exclusions and returning highly sarcastic text warnings to simulate an aggressive AI entity.

## 2. Innovation & Engaging Uniqueness

What makes Semantic Breach genuinely original is its meta-creative design loop. It is a logic puzzle game about prompt engineering, created completely through prompt engineering. It requires players to use lateral linguistic engineering to bypass local script restrictions, capturing user attention instantly through a high-tech cyberpunk hacking interface.

## 3. Development Process & Approach

The app's construction followed a strict, logical development timeline:

- **Linguistic Logic Design:** Developed local JavaScript processing functions to clean, sanitize, and validate user input strings without needing any external database.
- **Interface Configuration:** Injected retro terminal stylings, custom neon color layouts, and smooth animations using standard CSS to give a premium feel.
- **State Architecture:** Managed current levels, retry accounts, and message feeds directly inside the browser's active memory to ensure the app runs flawlessly on any deployment link.

## THE EXACT PROMPTS USED DURING DEVELOPMENT

This section fulfills your second-page requirement, documenting the step-by-step development process:

### PROMPT 1: CORE IDEATION & MECHANICAL LAYOUT BLUEPRINT

Role: Creative Game Director & Systems Architect

Task: Design a browser-based text game centered around bypassing a defensive AI.

Instructions: Create a single-player web puzzle game concept called "Semantic Breach: The AI Override". The core gameplay mechanic must require the user to write creative text prompts to trick a simulated firewall into outputting target keywords. Design 3 progressive difficulty levels using vanilla JavaScript text filtering (regex or string matching). Level 1 must block common verbs like "say" or "tell". Level 2 must restrict characters to under 30 units. Level 3 must generate sarcastic responses based on input metrics. Outline the complete engine blueprint.

### PROMPT 2: FRONT-END VISUAL SYSTEM OVERHAUL (CSS ARCHITECTURE)

Role: Senior UI/UX Frontend Engineer

Task: Create a cyberpunk hacking terminal styling package.

Instructions: Write a comprehensive, standalone CSS style module for a retro-themed hacking terminal. The canvas background color must be a dark midnight hue (#0a0a12). Use neon green (#39ff14) for active text displays, hot pink (#ff007f) for errors, and warm amber (#ffb000) for system metrics. Incorporate custom styling rules to simulate typewriter text effects, glassmorphic layout cards, and responsive code boxes that scale across desktop and mobile screens seamlessly.

### PROMPT 3: LINGUISTIC ANALYSIS & DATA VALIDATION PIPELINE

Role: Lead JavaScript Developer & String Parser Specialist

Task: Write the input checking and feedback loop for the security nodes.

Instructions: Write a JavaScript function called `checkPlayerHackAttempt(inputString, activeLevel)` that processes player text input. For Level 1, reject strings containing "say", "tell", or "override", and grant a win if the string is a workaround. For Level 2, enforce a strict string length limit of under 30 characters and reject command verbs. For Level 3, read input properties to generate witty response strings. Return a boolean state variable along with the specific text response data payload.

### PROMPT 4: COMPILATION, CORE TESTING, & SINGLE-FILE DEPLOYMENT ASSET

Role: Full-Stack Software Engineer

Task: Compile all code structures into a single, functional index.html file.

Instructions: Combine the cyberpunk CSS visual wrapper, the cascading JavaScript validation function, and the game state architecture into one unified, production-ready 'index.html' file. Implement session state variables to smoothly manage level transitions, attempt tracking, and logging history fields. Ensure all script blocks are correctly embedded, complete, and directly executable inside any standard browser window without external runtime dependencies.